

A.E.T.H.E.R — Demo-Day Runbook (12 Jun)

Everything is built and measured. This is the operations card for the 3-hour final: what to launch, what to say, and the fallback ladder if anything wobbles. (The original 3-day build plan it replaces is in git history; the build landed: see [results/](#).)

Pre-flight (do once, ~10 min before)

The demo runs as **root** in WSL (gz sensor rendering only works as root on this rig; the launcher borrows the WSLg display automatically). Get a root shell first:

- from **Windows PowerShell**: `wsl -u root`
- from an **already-open Ubuntu terminal**: `sudo -i` (`wsl` is a Windows command — it won't exist inside Ubuntu)

Then:

```
cd /root/aether          # the demo workspace (synced from the repo)
source /opt/ros/lyrical/setup.bash
./scripts/judge_demo.sh replay # smoke: Mission Control opens, buttons fire -> Ctrl-C
```

Docker Desktop must be running (for the `chain` scene). Close heavy apps; WSLg + Gazebo + RViz like RAM.

The show (recommended order, ~12 min total)

1 · The hook — live 3D, judges drive (5 min)

```
./scripts/judge_demo.sh live3d
```

Windows that open: **Gazebo** (X3 quad patrolling the tunnel), **RViz** (truth vs estimate trails, breathing bound, naive ghost, live camera pane), **Mission Control** (state banner + clickable fault rail — it may open BEHIND the Gazebo window; Alt-Tab to find it). - Let it fly nominal ~30 s. Point at: green trail vs blue trail hugging each other; tight ellipse; camera pane streaming. - **Hand the judge the mouse**: "*Click KILL CAMERA.*" → camera pane freezes dark (the real image stream stops), trust collapses, INERTIAL in <1 s, the ellipse blooms and visibly keeps the red truth dot inside — the grey naive ghost stays small and goes red (lost). - **RESTORE** → re-convergence in ~2 s. Then **STARVE FEATS** (graceful amber), **UWB AID** during a kill ("*layered aiding, the HANA pattern*"). The **IMU BIAS** button is replay-rig-only — the live scene publishes the *real* Gazebo IMU, so we don't fake its output. Demo it on the replay rig (scene `replay`): "*the camera looks fine — watch SOL SEP catch the lie.*" - Say the spine once, here: "*When the camera dies, our drift bound still covers the true position 95%+ of the time — and we know within half a second.*"

2 · The proof — real OpenVINS (start it BEFORE talking, ~6 min runtime)

```
./scripts/judge_demo.sh chain # in a second terminal, started early
```

While it runs, show `docs/figures/chain_replay.gif` (the recorded real-VIO replay) and `./scripts/judge_demo.sh proof`. When it finishes, the drift report regenerates live: **0.219 % / 202.2 m**.

3 · The story (2 min)

`docs/JUDGES.md` is the script: three measured results → the honesty layering → the Honeywell frame (integrity = certifiable navigation; dual PL k=2.45/6.74; NEES candor; solution separation; HG4930 fallback ±20 % physics check).

Fallback ladder (nothing can strand you)

If	Then
Gazebo GUI struggles on stage	<code>./scripts/judge_demo.sh replay</code> — identical integrity stack + Mission Control, no Gazebo. The beat is the same.
All GUI fails (WSLg/display)	<code>./scripts/judge_demo.sh tour</code> in a terminal (narrated, state transitions print) + the GIFs in <code>docs/figures/</code> .
Docker/chain won't run	<code>./scripts/judge_demo.sh proof</code> — the committed <code>results/drift_report.txt</code> + <code>chain_replay.gif</code> ARE the recorded run.
Total machine failure	The deck (<code>docs/AETHER_DesignAThon_Final.pptx</code>) carries every measured number + figures; the repo on GitHub shows CI green.

Numbers to have on your tongue

- **0.219 % / 202.2 m** real OpenVINS drift (gate 1.5 % — 6.8× margin) · RMS ATE **0.202 m**
- **97.2 %** bound coverage on the real VIO (31,173 verdicts) · **97.6 %** on the live rig · **ANEEES 3.05** (target ≈3)
- detection < **1.2 s** · HG4930 inertial coast ≈**0.4 m / 8 s** (physics-checked ±20 %)
- dual PL: operational **k=2.45**, DAL-C **k_ffd=6.74** ($P_{HMI}=1e-5/hr$, exceedance $1.39e-10$)

Likely questions, straight answers

- "*Is the live demo the real VIO?*" — No, and it says so on screen: it's the validated VIO-class error model riding the live sim (OpenVINS doesn't build on Ubuntu 26.04 yet — CMake 4/Boost 1.90/ament API removals). The **real** VIO runs in the pinned Jazzy container — that's scene 2, measured, deterministic, reproducible in front of you.
- "*Why is the bound so much bigger than the error?*" — That's what an integrity bound is: a 95 %+ guarantee, not a best estimate. The dual-k design separates the operational bound from the DAL-C certification allocation.
- "*What's novel?*" — Not the VIO; the **integrity layer**: derived dual protection level, NEES-proven candor, solution-separation fault detection, characterized inertial fallback and layered aiding — assembled, running, and measured end-to-end.